

Callbacks and Timers

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Single-Threaded Execution

- JavaScript runs in **one thread**, one thing at a time
- Each piece of code must **finish** before the next starts
- Currently running code lives on the **call stack**
- Async behavior is what makes this workable

The Host Environment

- The browser or Node.js provides extra features:
 - Timers (timeouts, intervals)
 - Network requests
 - DOM events (click, input, etc.)
- These are **not** part of the JS language; they are **Web/host APIs**
- The environment starts the work, then queues the callback when ready

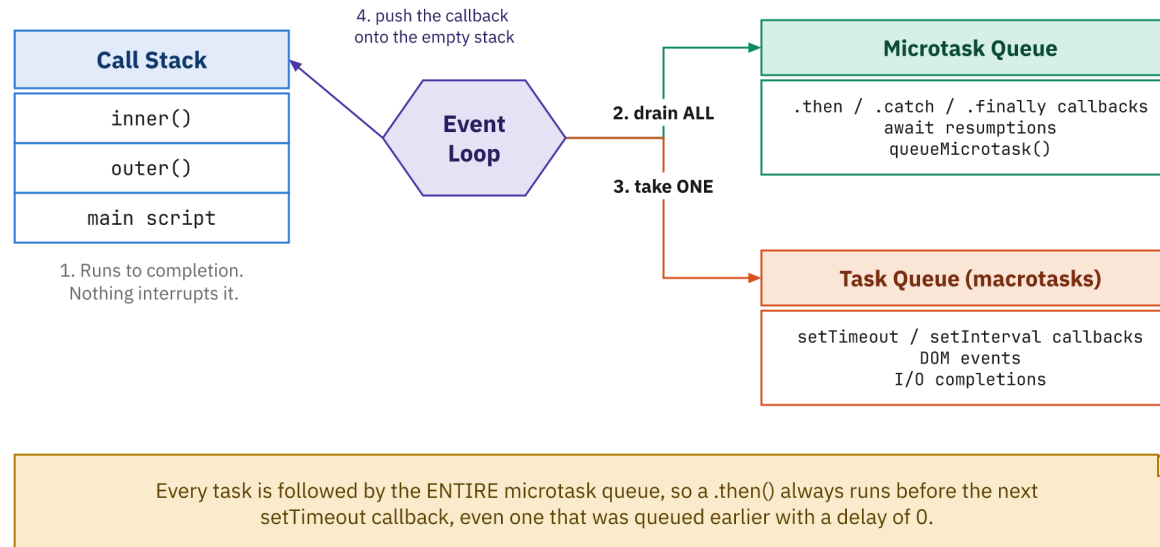
Tasks and Microtasks

- Two main queues feed the event loop:
 - **Task (macrotask) queue**: timers, UI events, network callbacks
 - **Microtask queue**: promises, high-priority small tasks
- Key rule: after each task, run **all microtasks** before the next task
- This is why promise callbacks run **before** timeout callbacks

The Event Loop Cycle

The Event Loop

One thread. The stack must be empty before anything queued can run.



- Stack empties → drain **all** microtasks → take **one** task → repeat
- A `.then()` always beats a `setTimeout`, even one queued earlier

Blocking the Event Loop

```
setTimeout(() => console.log("timer - finally!"), 0);

const stop = Date.now() + 1000;
while (Date.now() < stop) {} // busy for a full second
console.log("blocked for 1000ms");
// blocked for 1000ms
// timer - finally!
```

- While the stack is busy, nothing queued can run
- In a browser the page freezes: no paint, no clicks, no scrolling

What Is a Callback?

- A function you **pass to another function** to be called later
- Core to how JavaScript does asynchronous work
- Can be **synchronous** (runs immediately) or **asynchronous** (runs later)

```
function greet(name) {  
  console.log("Hello, " + name);  
}  
function doTwice(callback) {  
  callback("Alice");  
  callback("Bob");  
}  
doTwice(greet); // synchronous callback
```

Asynchronous Callbacks

- The callback runs **later**, via the event loop

```
console.log("Start");
setTimeout(function () {
  console.log("Inside timeout");
}, 1000);
console.log("End");
// Start
// End
// Inside timeout  <- runs later
```

- `setTimeout` registers the callback and moves on

Callbacks and DOM Events

- Event listeners are callbacks too

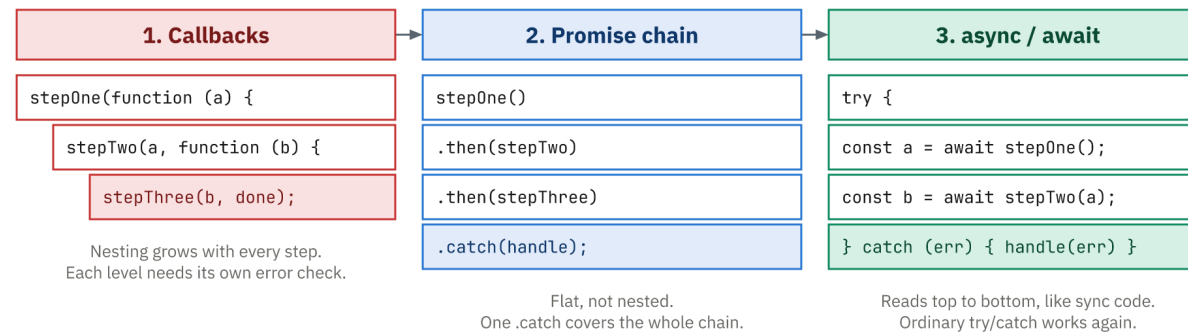
```
button.addEventListener("click", function () {  
  console.log("Button was clicked!");  
});
```

- You register the callback; the browser listens outside the JS engine
- On a click, the browser queues the callback
- The event loop runs it when the call stack is free

Callback Hell

The Async Progression

The same three dependent steps, written three ways. Only the shape of the code changes.



All three do the same work on the same event loop. async/await is not a new mechanism: an async function returns a promise, and every await is a .then() the engine writes for you.

- Nesting grows with every step, and each level needs its own error check
- Motivates **promises** and `async / await`: same event loop, better shape

setTimeout: Run Once, Later

- `setTimeout(callback, delay)` runs a function **once** after at least `delay` ms

```
console.log("start");
setTimeout(() => {
  console.log("later");
}, 1000);
console.log("end");
// start
// end
// later  <- ~1 second later
```

- The delay is a **minimum**, not a guarantee

Arguments and Canceling

- Extra arguments after the delay pass to the callback

```
setTimeout((name) => {  
  console.log("Hello, " + name);  
}, 500, "Alice"); // Hello, Alice
```

- `setTimeout` returns an **id**; cancel with `clearTimeout`

```
const id = setTimeout(() => console.log("never"), 1000);  
clearTimeout(id);
```

setInterval: Run Repeatedly

- `setInterval(callback, delay)` repeats until you stop it

```
let count = 0;
const id = setInterval(() => {
  count++;
  console.log("tick " + count);
  if (count === 3) clearInterval(id);
}, 1000);
// tick 1, tick 2, tick 3 (then stops)
```

- **Always keep the id:** an uncleared interval never lets the program exit

queueMicrotask: Run Very Soon

- Schedules on the **microtask queue**, same one promises use
- Runs after current code, but **before any timer**

```
console.log("1: sync");
setTimeout(() => console.log("2: timeout"), 0);
queueMicrotask(() => console.log("3: microtask"));
console.log("4: sync");
// 1: sync
// 4: sync
// 3: microtask  <- before timers
// 2: timeout
```

Pitfall: Interval Drift

- `setInterval` does **not** guarantee even spacing; ticks drift
- For accuracy, self-correct with a `setTimeout` chain

```
function everySecond(callback) {  
  let expected = Date.now() + 1000;  
  function tick() {  
    const drift = Date.now() - expected;  
    callback();  
    expected += 1000;  
    setTimeout(tick, Math.max(0, 1000 - drift));  
  }  
  setTimeout(tick, 1000);  
}
```